

Final Project Proposal: A Hybrid Chess Engine for the FIDE & Google Efficient AI Challenge

Abby Kaur

Baskin School of Engineering
University of California, Santa Cruz
Santa Cruz, California 95064
Email: akaur79@ucsc.edu

Avni Gandhi

Baskin School of Engineering
University of California, Santa Cruz
Santa Cruz, California 95064
Email: avgandhi@ucsc.edu

Max Ratcliff

Baskin School of Engineering
University of California, Santa Cruz
Santa Cruz, California 95064
Email: mwratcli@ucsc.edu

Mann Malviya

Baskin School of Engineering
University of California, Santa Cruz
Santa Cruz, California 95064
Email: mmalviya@ucsc.edu

Abstract—This project follows the Kaggle [4] Challenge Implementation track and focuses on developing a custom chess engine capable of autonomously playing and evaluating chess games. The goal is to create an intelligent system that not only follows the rules of chess but also learns to make optimal strategic decisions through algorithmic and data-driven approaches using a hybrid ML approach. The planned methodology involves implementing a traditional minimax search with alpha-beta pruning as a foundational decision-making framework, then enhancing the engine’s performance using supervised and reinforcement learning on Kaggle’s public chess game datasets and/or the Lichess game database. The engine will be trained to evaluate board positions and predict high-probability moves based on historical game outcomes. Performance will be measured through self-play simulations, ELO-style rating comparisons, and move prediction accuracy. The final deliverables include a fully functional chess engine, accompanying Kaggle notebooks for data analysis and training, and a detailed report analyzing model performance and limitations. This project aims to bridge classic game theory with modern machine learning, demonstrating how hybrid ML methods can optimize decision-making in complex strategic environments.

I. INTRODUCTION

Artificial intelligence has played a major role in game-playing research, showing how computers can learn, plan, and make decisions in complex situations. Among all games, chess has been one of the most important testing grounds for AI progress. Chess has shown how powerful algorithms can become when they combine strategy with learning. This project aims to build on that tradition by developing a custom chess engine that blends classic search algorithms with modern machine learning. The goal is to create a system that can not only play legal moves but also adapt and improve based on real game data. This work is meaningful because the same principles that are used in chess solvers, decision-making under pressure, pattern recognition, and optimization are also used in real-world systems such as robotics, resource planning, and autonomous vehicles. By using Kaggle’s data-

driven environment, the project will test these ideas in a competitive and measurable way, helping bridge theory with hands-on experimentation.

II. PROJECT PROPOSAL

We have chosen **Track 2** (Kaggle Challenge Implementation) for our final project. Our team will follow the guidelines outlined in the FIDE & Google Efficient Chess AI Challenge on Kaggle (<https://www.kaggle.com/competitions/fide-google-efficiency-chess-ai-challenge>) [4]. This competition focuses on finding out how we can **develop an intelligent chess engine effectively which is capable of evaluating positions and predicting optimal moves using limited computational resources**, time and a randomized opening. Our goal is to design and train a model that balances performance, accuracy, and efficiency, while we explore different approaches such as neural network-based evaluation functions combined with classic search algorithms.

III. PROPOSED METHODS

Building a chess engine can be broken up into 3 main problems: Board Representation & Legal Move Generation, Engine Architecture & Move Decisions, and Machine Learning for Evaluation & Search Guidance

A. Board Representation & Legal Move Generation

The choice of board representation is a critical design decision in a chess engine, involving a fundamental trade-off between execution performance and implementation complexity. A representation must encode all information necessary to generate legal moves and evaluate the position.

1) *Alternate Board Representations*: In planning our architecture, we explored several standard approaches in detail:

- **Bitboards [1], [6]**: This is the modern standard for high-performance engines. A bitboard architecture represents the board using a set of 64-bit integers, where each bit maps directly to a square on the board. A full position is described by multiple bitboards (e.g., one for white pawns, one for black knights, one for all occupied squares). The primary advantage of this method is its high speed. Complex questions like “which squares are attacked by a rook on e4?” can be answered with a few bitwise operations (shifts, masks, and logical ANDs/ORs), allowing the engine to analyze millions of positions per second. Advanced techniques like **rotating bitboards** use clever hashing schemes to handle move generation for sliding pieces (rooks, bishops, queens) with minimal computation.
- **0x88 Method [1]**: An efficient array-based representation that uses a 1×128 (or 16×8) array. Only half of this array corresponds to the legal 64 squares of the board; the other half acts as an “illegal” zone. This allows a square’s index to be validated with a single bitwise AND operation (e.g., if (square & 0x88) != 0). A non-zero result means the move is off-board, eliminating the need for slower boundary checks. Furthermore, the difference between two square indices in this scheme can uniquely identify if they are aligned on the same rank, file, or diagonal, which is useful for move generation and evaluation.
- **Piece lists [1]**: A classic, intuitive method that maintains a simple array for each piece type, storing its location and other attributes. While easy to conceptualize and iterate over (e.g., “for each of white’s pawns...”), it is computationally inefficient for the spatial queries required by a modern search algorithm. Answering the question “what piece is on square e4?” would require scanning multiple lists, making it much slower than direct-lookup methods like arrays or bitboards.

2) *Chosen Board Representation*: For this project, we will utilize the *python-chess* library [2] for all board representation and move generation. While a high-level library is not as performant as a native bitboard implementation, it provides a correct and fully-featured API. This strategic decision allows us to abstract away the significant complexity of low-level chess logic and focus our research efforts on designing the search algorithm and developing the ML-based evaluation function. A future direction for this project to achieve competitive performance would be to re-implement the core engine logic using a native bitboard representation in numpy or a lower-level language like C++.

B. Engine Architecture & Move Decisions

Our engine’s design is a hybrid model that combines a classical, high-performance search algorithm with a modern, machine learning-based evaluation and guidance system.

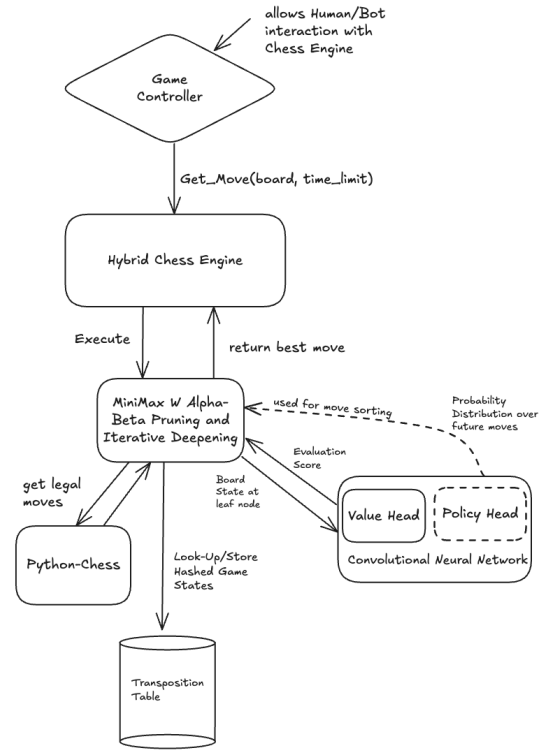


Fig. 1. High-level architecture of our hybrid chess engine. The alpha-beta search algorithm orchestrates the process, using the *python-chess* library for game state management and move generation, while querying our neural network for strategic evaluation and guidance.

1) *Core Search Algorithm*: The foundation of our engine is a highly optimized alpha-beta search algorithm [3] responsible for the logical exploration of the game tree. It will be implemented with several key strategies:

- **Minimax with Alpha-Beta Pruning [3]**: A recursive, depth-first search that assumes optimal play from both players. We will implement alpha-beta pruning with a ML-based evaluation function to dramatically reduce the search space by eliminating branches that cannot influence the final decision.
- **Iterative Deepening**: The engine will perform successive depth-limited searches (depth 1, then 2, then 3, etc.) until the time/resource limit is approached. This ensures a move is always ready and uses results from shallower searches to improve move ordering, significantly enhancing the effectiveness of alpha-beta pruning.
- **Transposition Tables**: To avoid re-analyzing identical game positions and improve the efficiency of iterative deepening, we will implement a caching mechanism using Zobrist hashing [1] to store previously computed evaluations.

C. Machine Learning for Evaluation and Guidance

To provide the strategic understanding necessary for high-level play, we will improve traditional, handcrafted evaluation heuristics with a learned neural network using a Residual

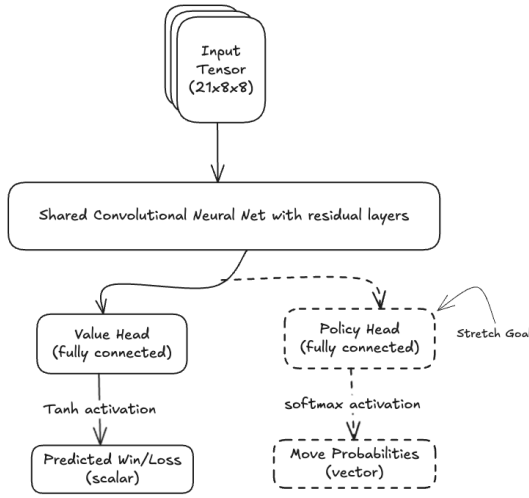


Fig. 2. Proposed neural network architecture. A shared convolutional body processes the board state, feeding into two distinct heads for position evaluation (Value Head) and move probability prediction (Policy Head). This modular design allows for phased implementation.

[9] CNN architecture [8], [11]. This model will serve as the engine’s “chess knowledge,” trained on a massive dataset of games to recognize complex patterns and assess positional strength. The network will be designed with a modular architecture, in which we will focus efforts on a Value Head, with the potential of pursuing a dual headed policy informed approach if time allows.

1) *Input Tensor Representation*: A critical step is converting the board state into a numerical format suitable for a CNN. We will transform the *python-chess* board object into a $21 \times 8 \times 8$ tensor. This “image-like” representation provides a strong spatial inductive bias, allowing the CNN to effectively learn local patterns. The 21 channels are structured as follows:

- **Piece Planes (12 Channels)**: Twelve 8×8 binary planes represent the location of each piece type for each color (e.g., a plane for White Pawns, a plane for Black Knights). This provides the fundamental spatial layout.
- **Game State Planes (9 Channels)**: Nine 8×8 planes encode non-spatial game rules and history. These include a channel for the side to move, four channels for castling rights, one channel for the en passant square, a channel for the fifty-move clock, a channel indicating the repetition count for the current position, and a final channel for the total move number, which serves as a proxy for the game phase (opening, midgame, or endgame). This ensures the network has complete spatial, temporal, and rule-based context.

2) *The Value Network*: The core of our ML model is a **value head**, which acts as a sophisticated evaluation function. Its goal is to predict the outcome of the game from the input tensor.

- **Architecture**: The network will use a deep convolutional [8], [11] body with residual connections [9], inspired by the successful AlphaZero [12] architecture. This body is

designed to efficiently process the spatial patterns of the board. The output of the convolutional layers will be fed into a fully-connected head that produces a single scalar value via a hyperbolic tangent (tanh) activation function.

- **Training**: The value head will be trained via supervised learning on the Lichess [5] database. We will sample millions of positions from games and label each position with the final game result (+1 for a white win, -1 for a black win, 0 for a draw). The network will be trained to minimize the Mean Squared Error between its prediction and the actual outcome.

3) *The Policy Network (Stretch Goal)*: As an enhancement to deepen our research, we will investigate the addition of a **policy head**. This component’s purpose is not to evaluate positions, but to guide the search algorithm more intelligently.

- **Architecture**: As shown in Fig. 2, the policy head will branch off from the same shared convolutional body as the value head. It will have its own fully-connected layers, terminating in a softmax activation function.
- **Output**: The policy head will output a probability distribution over the set of all possible moves. This vector will represent the network’s belief about which moves are most likely to be played by a strong player in the given position.
- **Integration for Move Ordering**: The output from this policy network will be used to implement intelligent **move ordering**. At each node in the search tree, we will query the policy network and sort the legal moves from most to least probable. By exploring the strongest moves first, we drastically increase the number of branches pruned by the alpha-beta algorithm.

D. Tools, Libraries, and Data

- **Programming Language**: Python
- **Libraries**: *python-chess* for game logic; *PyTorch* for neural network development; *NumPy* for data manipulation.
- **Data Source**: The Lichess Elite Database [5] will be used as the primary source for training our neural network due to its huge volume of high-quality, labeled game data.

IV. GOALS AND AIMS

Our objectives are defined by a primary, measurable goal of achieving a performant chess engine, and several secondary aims for analysis and future direction.

A. Primary Goal

The primary goal is to develop a functional hybrid chess engine, compare it’s performance to the winners of the FIDE & Google Efficient Chess AI Challenge, and achieve a stable ELO rating of at least **2000** on a public chess leaderboard by the final project deadline. Success is defined as creating a competitive agent that can consistently outperform baseline rule-based engines and older ML-based Engines.

B. Secondary Aims

- To implement and test the policy network for move ordering as a stretch goal, measuring the resulting increase in search depth and ELO rating.
- To conduct a comparative analysis of our final hybrid engine against a purely classical version (using a simple heuristic evaluation function) to quantify the performance gain from the ML value network.
- To perform an error analysis on the model's evaluations, identifying specific types of positions (e.g., tactical vs. strategic) where the network excels or struggles.
- To implement a faster and more efficient board representation using bitboards or 0x88 representation.
- To Explore how other Search and Move selection heuristics like Monte-Carlo Tree search [14] effects our models performance.

REFERENCES

- [1] "Board representation (computer chess)," *Wikipedia, The Free Encyclopedia*, Oct. 12, 2025. [Online]. Available: [https://en.wikipedia.org/wiki/Board_representation_\(computer_chess\)](https://en.wikipedia.org/wiki/Board_representation_(computer_chess))
- [2] N. Fiekas, "python-chess: a chess library for Python," *python-chess.readthedocs.io*, 2024. [Online]. Available: <https://python-chess.readthedocs.io/en/latest/>
- [3] "Min-Max Algorithm in Artificial Intelligence," *GeeksforGeeks*, Aug. 22, 2024. [Online]. Available: <https://www.geeksforgeeks.org/min-max-algorithm-in-artificial-intelligence/>
- [4] "FIDE / Google - Efficient Chess AI Challenge," *Kaggle*, 2025. [Online]. Available: <https://www.kaggle.com/competitions/fide-google-efficiency-chess-ai-challenge/overview>
- [5] "Lichess Elite Database," *database.lichess.org*, 2025. [Online]. Available: <https://database.lichess.org/>
- [6] "Bitboards," *The Chess Programming Wiki*. [Online]. Available: <https://www.chessprogramming.org/Bitboards>
- [7] "Bitboard," *Wikipedia, The Free Encyclopedia*, Sep. 28, 2025. [Online]. Available: <https://en.wikipedia.org/wiki/Bitboard>
- [8] "Convolutional neural network," *Wikipedia, The Free Encyclopedia*, Oct. 15, 2025. [Online]. Available: https://en.wikipedia.org/wiki/Convolutional_neural_network
- [9] K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," *arXiv preprint arXiv:1512.03385*, 2015.
- [10] B. Anumukonda, "Building a Chess Engine with Reinforcement Learning," *bhaswanth-a.github.io*, May 28, 2021. [Online]. Available: <https://bhaswanth-a.github.io/posts/chess-engine-rl/>
- [11] S. Gunasekaran, "Convolutional Neural Networks," *CMPM118 Course Materials*, 2024. [Online]. Available: https://github.com/SkyeGunasekaran/CMPM118-NCG/blob/main/edu-material/4_cnn.md
- [12] D. Silver, T. Hubert, J. Schrittwieser, *et al.*, "A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play," *Science*, vol. 362, no. 6419, pp. 1140-1144, Dec. 2018, doi: 10.1126/science.aar6404.
- [13] G. van der Hoorn, A. Pasquale, and the LeelaChessZero team, "Leela Chess Zero," *lczero.org*, 2018. [Online]. Available: <https://lczero.org/>
- [14] C. B. Browne, E. Powley, D. Whitehouse, *et al.*, "A survey of monte carlo tree search methods," *IEEE Transactions on Computational Intelligence and AI in Games*, vol. 4, no. 1, pp. 1-43, March 2012.